A Bayesian neural network for `dihiggs`

Hamiltonian Monte Carlo after Radford Neal: the module `hmc.py`
and the notebook `heft_train_bnn.ipynb`

Plan, for approval before any code is written

October 29, 2026

Abstract

This plan describes how to turn the `HEFTNet` model of the di-Higgs cross section into a *Bayesian neural network* (BNN) in the sense of Radford Neal [1, 2]. Instead of a single best set of parameters, a BNN is an ensemble of parameter sets drawn from the posterior density of the parameters given the training data. The draws are made with Hamiltonian Monte Carlo (HMC), implemented from scratch in a new, model-agnostic Python module, `hmc.py`, which uses PyTorch only to compute gradients. The module finds a model's parameters by itself, groups them as Neal does, samples them, and writes each sampled parameter set to its own `params_NNNN.pth` file in a run folder. A companion class, `BNN`, reads those files back and computes the posterior mean of the network output, its standard deviation, quantiles and intervals. The module starts life in `dihiggs/` and is written so that it can move to `drwho` unchanged. A new notebook, `code/heft_train_bnn.ipynb`, in the style of `heft_train.ipynb`, drives it. Nothing will be written until you approve; the decisions needed are collected in section 10.

The proposal in brief

1. **What is sampled.** The posterior density $p(\omega, \tau, b \mid \mathcal{D})$ of the 3,944 network parameters ω, the prior precisions τ of Neal's 16 parameter groups, and the noise scale b.
2. **How.** HMC for ω (gradients from PyTorch autograd), alternated with exact Gibbs draws of τ and b, as in Neal's own software.
3. **Likelihood.** By default a Laplace likelihood, the probabilistic counterpart of the L1 loss we already train with. Its most probable point is exactly our current training with weight decay added.
4. **Start.** From the trained L1 model (a *warm start*), so that the chain begins where the posterior mass is, instead of spending most of the computing time getting there.
5. **Output.** `runs/BNN_<likelihood>/samples/params_0001.pth`, `params_0002.pth`, ..., each an ordinary `state_dict` that `HEFTNet().load()` reads, so the existing test and export notebooks work with any single sample.
6. **Validation first.** The sampler is checked on a toy problem whose posterior is known exactly before it is let loose on `HEFTNet`.

Contents

1 What will be delivered **2**

2 The Bayesian neural network **3**
2.1 From one best fit to an ensemble . . . 3
2.2 The prior, following Neal . . . 3
2.3 The likelihood . . . 4

3 Hamiltonian Monte Carlo, step by step **5**
3.1 The idea . . . 5
3.2 Tuning: step size, trajectory length, mass matrix . . . 7

4 Applying HMC to `HEFTNet` **7**
4.1 Warm start . . . 7
4.2 Cost . . . 7

5 The module `hmc.py` **8**
5.1 Design . . . 8
5.2 Finding the parameters automatically . . . 8
5.3 The classes and their methods . . . 10
5.4 The run folder . . . 10

6 The notebook `heft_train_bnn.ipynb` **11**

7 Validation **11**
7.1 The sampler on problems with known answers . . . 11
7.2 Diagnostics for the `HEFTNet` run . . . 11
7.3 What counts as success . . . 12

8 Moving the module to `drwho` **12**

9 Not in this plan **12**

10 Decisions for you **13**

1 What will be delivered

file	what it is
`dihiggs/hmc.py`	the module: parameter discovery, priors, likelihoods, the posterior, the HMC sampler, the `BNN` ensemble and convergence diagnostics. Depends only on `torch`, `numpy` and the Python standard library (plus `pyyaml`, already a dependency).
`code/heft_train_bnn.ipynb`	the notebook that samples the `HEFTNet` posterior and plots the results, in the style of `heft_train.ipynb`
`code/hmc_toy.ipynb`	(if you agree, section 10) checks of the sampler on two small problems, one with an exactly known posterior
`docs/bnn/bnn_plan.pdf`	this document

Every line of `hmc.py` will be commented for readers who know Python and statistics but not PyTorch's internals: what each tensor holds, its shape, and why each step is done. Constructs that tend to confuse (in-place operations, `torch.no_grad()`, `detach()`, slicing of flat vectors, generators) will each get a short explanation where they are first used.

2 The Bayesian neural network

2.1 From one best fit to an ensemble

At present, training finds one parameter vector $\boldsymbol{\omega}^*$ by minimising the empirical risk, and the cross section is predicted as $f(\boldsymbol{x};\boldsymbol{\omega}^*)$, with $\boldsymbol{x} = (m_{hh}, \kappa_\lambda, c_t, c_{tt}, c_{ggh}, c_{gghh})$. Neal's idea [1] is to treat $\boldsymbol{\omega}$ like any other unknown parameter in a Bayesian analysis. Bayes' theorem gives the posterior density

$$p(\boldsymbol{\omega} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{\omega})\, \pi(\boldsymbol{\omega})}{p(\mathcal{D})}, \qquad \mathcal{D} = \{(\boldsymbol{x}_i, t_i),\ i = 1, \ldots, N\}, \tag{1}$$

where $p(\mathcal{D} \mid \boldsymbol{\omega})$ is the likelihood, $\pi(\boldsymbol{\omega})$ the prior and t_i the POWHEG cross section in a bin. The prediction at a new point $\boldsymbol{x}$ is then not a single number but the posterior mean, with its spread:

$$\bar{f}(\boldsymbol{x}) = \int f(\boldsymbol{x};\boldsymbol{\omega})\, p(\boldsymbol{\omega} \mid \mathcal{D})\, d\boldsymbol{\omega} \;\approx\; \frac{1}{K}\sum_{k=1}^{K} f(\boldsymbol{x};\boldsymbol{\omega}_k), \qquad \boldsymbol{\omega}_k \sim p(\boldsymbol{\omega} \mid \mathcal{D}). \tag{2}$$

The integral over 3,944 dimensions cannot be done directly. It is approximated by an average over K parameter vectors $\boldsymbol{\omega}_1, \ldots, \boldsymbol{\omega}_K$ *sampled* from the posterior. Those samples are what the HMC sampler produces, and what is stored in the `params_NNNN.pth` files. In the same way, the standard deviation of $f(\boldsymbol{x};\boldsymbol{\omega}_k)$ over k measures how well the data determine the function at $\boldsymbol{x}$, and its quantiles give credible intervals.

What the band means, and what it does not

The spread of $f(\boldsymbol{x};\boldsymbol{\omega}_k)$ is the *epistemic* uncertainty: how much the function could vary given the finite, noisy training data and the flexibility of the network. We expect it to be small where the training data are dense and larger where they are sparse or where the model struggles, e.g. just above threshold. It is *not* an uncertainty on POWHEG, nor on the HEFT calculation itself. With about 274,000 training rows, the posterior will be narrow, and the band may well be smaller than the POWHEG noise almost everywhere. Finding out where it is not is one of the useful results of the exercise.

2.2 The prior, following Neal

Neal groups the parameters of a network by role and gives each group g its own Gaussian prior, whose width is itself unknown [1, ch. 2 and 3]:

$$\boldsymbol{\omega}_g \mid \tau_g \sim \mathcal{N}\left(0,\ \tau_g^{-1} I\right), \qquad \tau_g \sim \mathrm{Gamma}(\alpha_g, \beta_g). \tag{3}$$

Here τ_g is the *precision* (one over the variance) of group g, and α_g, β_g are the shape and rate of its Gamma prior. Such a *hierarchical* prior lets the data decide how large the weights of each layer should be, instead of fixing that by hand. In `HEFTNet` there are 16 groups, one per parameter tensor (table 1); the module finds them automatically (section 5.2).

group (PyTorch name)	role	size	default prior scale σ_g
`P.0.weight`, `P.0.bias`	input (m_{hh}) $\to$ hidden 1	25 + 25	$1/\sqrt{1}$, 1
`P.2.weight`, `P.2.bias`	hidden 1 $\to$ hidden 2	625 + 25	$1/\sqrt{25}$, 1
⋮	⋮ (hidden layers 3 to 6)		
`P.12.weight`, `P.12.bias`	hidden 6 $\to$ 23 outputs (tanh)	575 + 23	$1/\sqrt{25}$, 1
`Q.weight`, `Q.bias`	m_{hh} $\to$ 23 exponents	23 + 23	1, 1
16 groups		3,944	

Table 1: The parameter groups of `HEFTNet`. The indices 0, 2, ..., 12 are the positions of the `nn.Linear` layers inside `nn.Sequential`; the odd positions hold the activation functions, which have no parameters. The proposed default prior mean of τ_g is $1/\sigma_g^2$, with $\sigma_g = 1/\sqrt{\text{fan-in}}$ for weights (Neal's scaling, which keeps the prior on the function sensible as the width grows [1, ch. 2]) and $\sigma_g = 1$ for biases.

Why the hyperparameters can be drawn exactly (Gibbs sampling)

Given the current weights $\boldsymbol{\omega}_g$ (with n_g elements), the density of τ_g is

$$p(\tau_g \mid \boldsymbol{\omega}_g) \propto \underbrace{\tau_g^{n_g/2} e^{-\tau_g \|\boldsymbol{\omega}_g\|^2/2}}_{\text{Gaussian prior on } \boldsymbol{\omega}_g} \underbrace{\tau_g^{\alpha_g - 1} e^{-\beta_g \tau_g}}_{\text{Gamma prior}} = \tau_g^{(\alpha_g + n_g/2) - 1} \, e^{-(\beta_g + \|\boldsymbol{\omega}_g\|^2/2)\,\tau_g},$$

which is again a Gamma density, with shape $\alpha_g + n_g/2$ and rate $\beta_g + \|\boldsymbol{\omega}_g\|^2/2$. (A prior with this property is called *conjugate*.) A new τ_g can therefore be drawn exactly with `torch.distributions.Gamma`, with no tuning and no rejection. Neal's software alternates such Gibbs draws of the hyperparameters with HMC updates of the weights [1]; we do the same.

2.3 The likelihood

The likelihood says how the POWHEG value t_i scatters about the network's prediction $f_i = f(\boldsymbol{x}_i; \boldsymbol{\omega})$. Three choices will be offered through a switch, `LIKELIHOOD`:

`LIKELIHOOD`	$-\log p(t_i \mid f_i)$	counterpart	noise parameter
`'laplace'` (default)	$\lvert t_i - f_i \rvert / b + \log 2b$	L1 loss	one scale b (pb), Gibbs-sampled (inverse-Gamma)
`'gaussian'`	$(t_i - f_i)^2/2s^2 + \log s$	MSE loss	one variance s^2, Gibbs-sampled (inverse-Gamma)
`'poisson'` (experimental)	$(t_i - f_i)^2/2\sigma_i^2$	CHI2 loss	none: $\sigma_i^2 = w_j t_i$ known from the event count

The link with what we already do

With the Laplace likelihood and the prior of eq. (3), minus the log of the posterior is

$$U(\boldsymbol{\omega}) = \frac{1}{b}\sum_{i=1}^{N}|t_i - f(\boldsymbol{x}_i;\boldsymbol{\omega})| \;+\; \sum_g \frac{\tau_g}{2}\,\|\boldsymbol{\omega}_g\|^2 \;+\; \text{const} = \frac{N}{b}\left[\underbrace{\frac{1}{N}\sum_i |t_i - f_i|}_{\text{our L1 empirical risk}} \;+ \underbrace{\sum_g \frac{b\,\tau_g}{2N}\|\boldsymbol{\omega}_g\|^2}_{\text{weight decay}}\right].$$

So the most probable point of the posterior is the L1 training of `heft_train.ipynb` with a (tiny, since N is large) weight decay. The BNN does not replace that training: it explores the neighbourhood of its answer, with a width set by the data. That is why the trained L1 model is a natural starting point for the sampler (section 4.1).

One noise scale for cross sections that span seven decades

The Laplace and Gaussian likelihoods use a single noise scale b (or s) for all bins, just as L1 and MSE treat an error of 10^{-4} pb the same in every bin. The fitted b will be set by the large cross sections. The *epistemic* band from the spread of the $\boldsymbol{\omega}_k$ is still meaningful, but a *predictive* band that adds noise of size b will be too wide for small bins and too narrow for large ones. The physically correct likelihood is `'poisson'`, with the known variance $\sigma_i^2 = w_j t_i$ of a histogram of 40,000 events of weight w_j. That is the CHI2 loss which trained poorly, probably because its weights span ten decades. HMC will not cure that bad conditioning, so `'poisson'` is offered as an experiment, not as the default.

The Gibbs draw of the noise scale works exactly as for τ_g. For the Laplace likelihood with an inverse-Gamma prior $b \sim \text{InvGamma}(\alpha_b, \beta_b)$,

$$b \mid \boldsymbol{\omega}, \mathcal{D} \sim \text{InvGamma}\Big(\alpha_b + N,\;\; \beta_b + \sum_i |t_i - f_i|\Big),$$

and for the Gaussian, $s^2 \mid \boldsymbol{\omega}, \mathcal{D} \sim \text{InvGamma}(\alpha_s + N/2,\; \beta_s + \sum_i (t_i - f_i)^2/2)$.

3 Hamiltonian Monte Carlo, step by step

3.1 The idea

Write the posterior as a Boltzmann distribution, $p(\boldsymbol{\omega} \mid \mathcal{D}) \propto e^{-U(\boldsymbol{\omega})}$, with the *potential energy*

$$U(\boldsymbol{\omega}) = -\log p(\mathcal{D} \mid \boldsymbol{\omega}) - \log \pi(\boldsymbol{\omega} \mid \tau). \tag{4}$$

HMC [3, 2] pretends that $\boldsymbol{\omega}$ is the position of a particle moving on the surface U, and gives it a fictitious momentum $\boldsymbol{p}$, one component per parameter, with kinetic energy $K(\boldsymbol{p}) = \frac{1}{2}\boldsymbol{p}^{\mathsf{T}} M^{-1} \boldsymbol{p}$, where M is a "mass matrix" (diagonal here). The joint density of $(\boldsymbol{\omega}, \boldsymbol{p})$ is $\propto e^{-H(\boldsymbol{\omega},\boldsymbol{p})}$ with $H = U + K$, so $\boldsymbol{\omega}$ and $\boldsymbol{p}$ are independent and the marginal of $\boldsymbol{\omega}$ is the posterior we want. Each HMC update has three steps:

1. **Kick.** Draw a fresh momentum, $\boldsymbol{p} \sim \mathcal{N}(0, M)$.

2. **Glide.** Follow Hamilton's equations, $d\boldsymbol{\omega}/ds = M^{-1}\boldsymbol{p}$, $d\boldsymbol{p}/ds = -\nabla U(\boldsymbol{\omega})$, for a fictitious time $L\varepsilon$, using L *leapfrog* steps of size ε:

$$\begin{aligned} \boldsymbol{p} &\leftarrow \boldsymbol{p} - \tfrac{\varepsilon}{2}\nabla U(\boldsymbol{\omega}) && \text{(half step in momentum)} \\ \boldsymbol{\omega} &\leftarrow \boldsymbol{\omega} + \varepsilon\, M^{-1}\boldsymbol{p} && \text{(full step in position)} \\ \boldsymbol{p} &\leftarrow \boldsymbol{p} - \tfrac{\varepsilon}{2}\nabla U(\boldsymbol{\omega}) && \text{(half step in momentum)} \end{aligned}$$

   (consecutive half steps in momentum are merged, so each leapfrog step costs one gradient).

3. **Accept or reject.** Exact dynamics would conserve H. Leapfrog does so only approximately, so the end point $(\boldsymbol{\omega}', \boldsymbol{p}')$ is accepted with probability $\min\{1, e^{-\Delta H}\}$, $\Delta H = H(\boldsymbol{\omega}', \boldsymbol{p}') - H(\boldsymbol{\omega}, \boldsymbol{p})$. If it is rejected, the chain stays at $\boldsymbol{\omega}$ (and $\boldsymbol{\omega}$ is counted again).

Figure 1 shows why this is worth the trouble. Neural-network posteriors are full of long, narrow ridges. A random-walk sampler must take steps no larger than the width of the ridge, so it diffuses along it slowly. An HMC trajectory follows the ridge, because the gradient keeps pushing it back into the valley, and in one update travels a distance comparable to the length of the ridge. Along the way U and K exchange energy while H stays nearly constant, so the acceptance probability stays high.

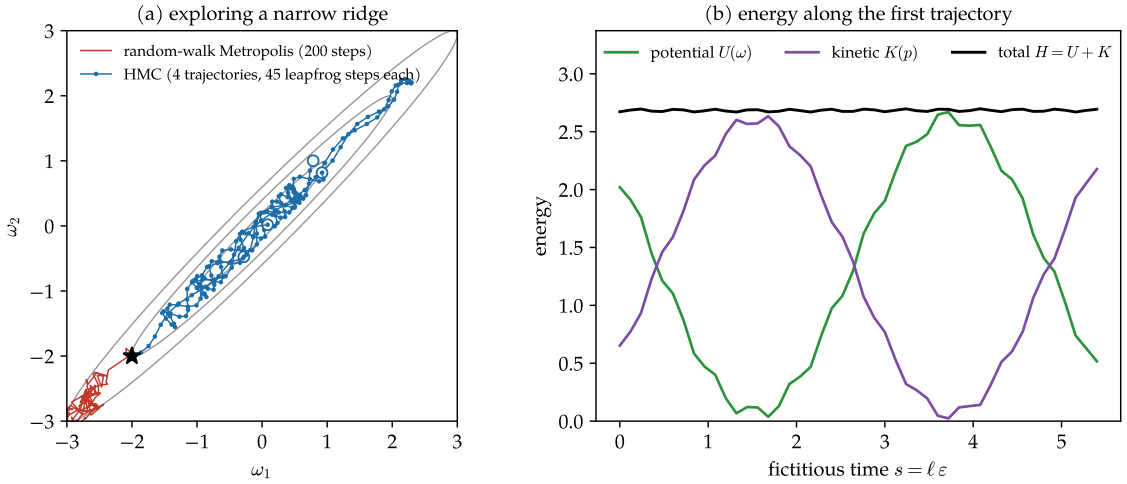


Figure 1: (a) Sampling a Gaussian with correlation 0.98, starting at the star. 200 random-walk Metropolis steps (red) barely move, while each of four HMC trajectories (blue; open circles are the end points) crosses much of the ridge. (b) Along one trajectory, potential and kinetic energy trade places while the total energy H is conserved to about 1%, so the end point is almost always accepted.

Why the result is exactly the posterior, and not an approximation

The leapfrog map has two properties that hold whatever ε is: it is *reversible* (run it again after flipping the sign of $\boldsymbol{p}$ and you return to the start) and it *preserves volume* in $(\boldsymbol{\omega}, \boldsymbol{p})$ space (each of the three sub-steps is a shear, which changes one set of coordinates by an amount that depends only on the other set). With these two properties, the accept/reject step makes the chain satisfy detailed balance with respect to e^{-H} exactly [2]. A large ε only lowers the acceptance rate; it does not bias the samples. Neither property needs U to be smooth, so the kinks of $|t - f|$ and of the ReLU are allowed; they merely make ΔH a little larger.

3.2 Tuning: step size, trajectory length, mass matrix

HMC has three settings. They are adjusted during a *warm-up* phase, whose samples are discarded, and then frozen, because changing them while sampling would break the Markov property.

- **Step size ε.** Too large: ΔH is large and most trajectories are rejected. Too small: each trajectory moves a short distance. During warm-up, ε is adjusted by *dual averaging* [4] so that the average acceptance rate reaches a target (default 0.8): raised when the acceptance is above target, lowered when below, with an averaging that makes it settle. In the sampling phase each trajectory uses ε times a random factor between 0.9 and 1.1 ("jitter"), as Neal recommends, to avoid trajectories that happen to return to their start.
- **Number of leapfrog steps L.** Fixed by the user (switch `N_LEAPFROG`), also with a small random jitter. The diagnostics of section 7.2 tell whether it is long enough.
- **Mass matrix M.** The leapfrog step in ω is $\varepsilon M^{-1}\boldsymbol{p}$. If some parameters are far more tightly constrained than others, one ε cannot suit all of them. Setting M^{-1} to the posterior *variance* of each parameter, estimated from the second half of the warm-up, rescales every parameter to unit width [2]. This is a diagonal matrix of 3,944 numbers.

What HMC requires of the problem

1. $U(\omega)$ and $\nabla U(\omega)$ can be computed (almost everywhere) for any ω. PyTorch autograd supplies the gradient.
2. The model is a *deterministic* function of its inputs and parameters: no dropout and no batch normalisation. The module checks this.
3. **Every gradient uses all N training rows.** A gradient from a random mini-batch is noisy, and noisy gradients destroy the conservation of H on which the accept/reject step relies. (Stochastic-gradient variants exist [6], but they sample the posterior only approximately and are not Neal's method.) The sum over rows will be computed in chunks to bound memory; the sum of the chunks is exact.

4 Applying HMC to `HEFTNet`

4.1 Warm start

Neal starts his chains from small random weights and lets them find the region of high posterior density. With 274,000 rows, a full-data gradient per leapfrog step and a narrow posterior, that "burn-in" would consume most of the computing time. The proposal is therefore to start from the parameters of a trained L1 model (switch `WARM_START`, default `runs/L1Loss_ids/dihiggs_params.pth`), which is close to the posterior mode (section 2.3). The hyperparameters start at values computed from those weights: $\tau_g = n_g / \|\omega_g\|^2$, and b equal to the mean absolute residual. A short warm-up then lets the chain settle into the posterior proper. Setting `WARM_START = None` reproduces Neal's start from the prior, for comparison.

4.2 Cost

Your full training run, 200,000 iterations of 2,048 rows in about 42 minutes on a CPU, implies about 6 μs per row for a forward and backward pass. A full-data gradient therefore costs at most about

1.7 s on that CPU, and a trajectory of $L = 100$ leapfrog steps about 3 minutes. 1,000 trajectories would take two days on the CPU (an upper bound, since large batches use the CPU more efficiently than batches of 2,048). On a GPU, where the 274,000 rows fit in one pass, a gradient should take milliseconds, and the same run an hour or less. The notebook will *time* one gradient before sampling and print the predicted run time, so you can choose `N_LEAPFROG` and `N_SAMPLES` knowingly. A switch `DATA_FRACTION` uses only a fraction of the training spectra (whole spectra, not rows), for quick tests; the posterior is then wider, as it should be with less data.

Things that can go wrong, and what the module does about them

1. **Bad conditioning.** Neural-network posteriors have directions that are constrained a million times more tightly than others. A diagonal mass matrix removes differences of scale between parameters but not correlations among them. If the acceptance is good but the samples are strongly autocorrelated, L must grow. Large studies of full-data HMC for networks [5] needed long trajectories; we should expect the same.
2. **Precision.** U is a sum of 274,000 terms of order 1 or more, while the accept/reject step needs ΔH to about 0.01. In single precision (about 7 digits) the sum would be at the edge of that. The model will run in single precision, but U, H and the chunk sums will be accumulated in double precision.
3. **Divergences.** If ΔH exceeds 1,000, the trajectory has gone unstable (e.g. e^Q in `HEFTNet` overflowed). It is rejected, counted, and reported; many divergences mean ε is too large.
4. **Symmetries.** Permuting the hidden units of a layer, or rescaling a ReLU unit's input weights by c and its output weights by $1/c$, leaves f unchanged. Individual weights therefore need not converge, and are not monitored; only quantities that matter, U, the hyperparameters and predictions at chosen points, are.

5 The module `hmc.py`

5.1 Design

Figure 2 shows the pieces and how data flow between them. Each piece is a small class with one job. None of them knows anything about di-Higgs physics: the model, the data and the likelihood are handed in. That is what will let the file move to `drwho` unchanged.

5.2 Finding the parameters automatically

Every PyTorch model is an `nn.Module`, and every `nn.Module` can list its own parameters, with their names, through `named_parameters()`. That is all `ParameterView` needs; it never looks inside the model's code. For `HEFTNet` it finds the 16 tensors of table 1, by name and shape, and by default makes each tensor a group of Neal's prior. It also provides the two conversions HMC needs: from the model's tensors to one flat vector $\boldsymbol{\omega}$ of 3,944 numbers (the "position") and back. A sketch, with comments of the kind the final code will have:

```
class ParameterView:
    def __init__(self, model):
        self.model = model
        self.names, self.params = [], []
```

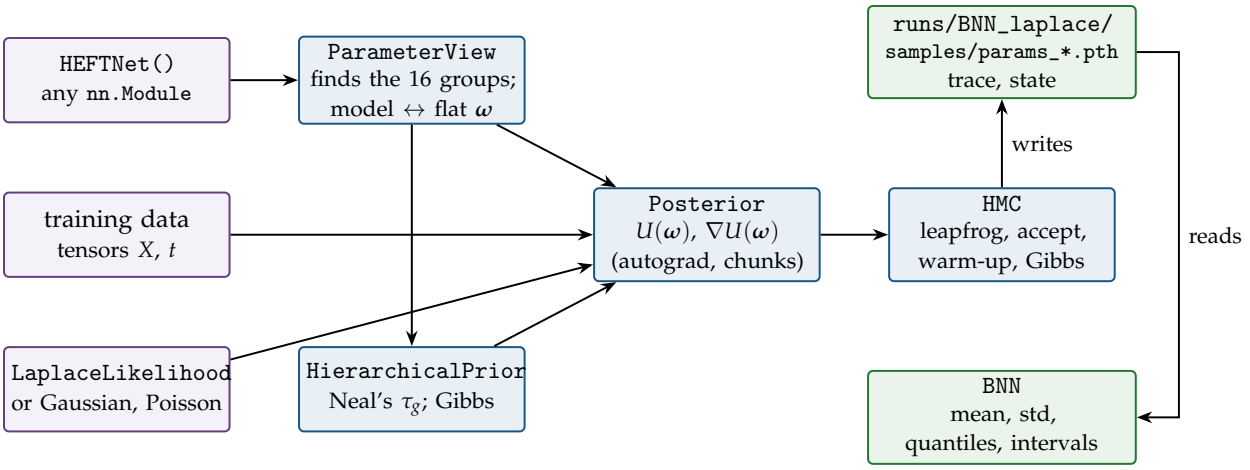


Figure 2: The classes of `hmc.py` (blue), what the user provides (purple) and what the sampler produces (green).

```
        # named_parameters() walks through every sub-module of the model and
        # yields (name, tensor) pairs, e.g. ('P.0.weight', tensor of shape
        # [25, 1]), ('P.0.bias', tensor of shape [25]), ..., ('Q.bias', [23]).
        for name, p in model.named_parameters():
            if p.requires_grad:           # skip parameters frozen by the user
                self.names.append(name)
                self.params.append(p)
        # numel() = number of elements of a tensor, e.g. 25*25 = 625.
        self.sizes = [p.numel() for p in self.params]
        # where each tensor sits inside the flat vector: a list of slices,
        # e.g. 'P.0.weight' -> slice(0, 25), 'P.0.bias' -> slice(25, 50), ...
        ends = np.cumsum(self.sizes)
        self.slices = [slice(e - n, e) for e, n in zip(ends, self.sizes)]

    def get(self):
        # Copy all parameters into ONE flat vector (the HMC "position").
        # detach() cuts the link to the autograd history; clone() makes an
        # independent copy, so changing it does not change the model.
        return parameters_to_vector(self.params).detach().clone()

    def set(self, w):
        # Write a flat vector back into the model's tensors, in place.
        vector_to_parameters(w, self.params)
```

The gradient ∇U is obtained the same way: compute U with the model's parameters set to ω, call `U.backward()`, and collect each parameter's `.grad` into a flat vector in the same order. Groups can also be chosen coarser (`grouping='all'`: one τ for every parameter) or by an explicit dictionary from names to groups.

What a model must satisfy. It is a `torch.nn.Module`; its `forward` takes a tensor of inputs and returns one prediction per row; it has no dropout or batch-normalisation layers (`ParameterView` inspects `model.modules()` and stops with an explanation if it finds one); and it can be rebuilt by calling its class with no arguments, so that `BNN` can recreate it. `HEFTNet` satisfies all four, as does `drwho`'s `FCNN`.

5.3 The classes and their methods

class	main methods
`ParameterView`	`get()`, `set(w)`, `grad()`, `groups`, `summary()` (prints the table of groups)
`HierarchicalPrior`	`neg_log(w)` $= \sum_g \tau_g \|\omega_g\|^2/2$; `gibbs(w)`: new τ_g; `from_weights(w)`: initial τ_g
`LaplaceLikelihood`, `GaussianLikelihood`, `PoissonLikelihood`	`neg_log(f, t)`; `gibbs(residuals)`: new noise scale (not for Poisson); `sample(f)`: simulated data, for predictive bands
`Posterior`	`U(w)`, `U_and_grad(w)`: sum over chunks of rows, in double precision; `time_gradient()`
`HMC`	`run(n_warmup, n_samples, thin)`; `resume()`; `acceptance_rate()`; internally `leapfrog()`, `trajectory()`, `adapt_step_size()`, `adapt_mass()`
`BNN`	`predict(x)` $\rightarrow$ tensor $[K, N]$; `mean(x)`, `std(x)`, `quantiles(x, q)`, `interval(x, cl)`, `predictive(x)` (adds the likelihood noise), `apply(fn, x)` (any function of a model, e.g. the coefficient functions `model.coeffs`), `len(bnn)`
functions	`autocorrelation(y)`, `ess(y)` (effective sample size), `rhat(chains)` (Gelman–Rubin)

For many points (e.g. all 274,000 training rows), `mean` and `std` are accumulated one sample at a time (Welford's running formulae), so the $K \times N$ table of predictions is never held in memory. A typical use:

```
from dihiggs import hmc
from dihiggs import heftnet as NN

bnn = hmc.BNN(NN.HEFTNet, 'runs/BNN_laplace')  # reads every params_NNNN.pth
mu  = bnn.mean(x)                   # posterior mean of sigma, one per row of x
sd  = bnn.std(x)                    # its posterior standard deviation
lo, hi = bnn.interval(x, cl=0.68)  # central 68% credible interval
A   = bnn.apply(lambda m, x: m.coeffs(x[:, 0]), x)  # samples of a_i(m_hh)
```

5.4 The run folder

file	contents
`runs/BNN_laplace/dihiggs_config.yaml`	every setting (`drwho Config`), the warm-start file and the training file's SHA-256, as in `heft_train`
`dihiggs_hmc_trace.csv`	one row per trajectory: iteration, phase (warm-up or sampling), accepted?, ΔH, U, ε, L, b, $\tau_1 \ldots \tau_{16}$
`dihiggs_hmc_state.pth`	checkpoint, rewritten every few minutes: current ω, τ, b, ε, M, iteration, random-number-generator states
`samples/params_0001.pth`, ...	one sampled parameter set each, a plain `state_dict`, readable by `HEFTNet().load(file)`
`samples/index.csv`	for each sample: file name, iteration, U, b, τ_g

A sample is about 16 kB, so 1,000 samples take 16 MB. Because of the checkpoint, a run interrupted by Colab, a time limit on the RTX 5090 cluster, or a closed laptop lid continues with `sampler.resume()` from where it stopped, and the result is the same as an uninterrupted run with the same seed. With `N_CHAINS` > 1, each chain gets its own sub-folder, `chain_1/`, `chain_2/`, ..., and its own seed; `BNN` pools them.

6 The notebook `heft_train_bnn.ipynb`

The notebook follows `heft_train.ipynb` cell for cell where it can (header with `Created/Updated` lines, Colab set-up cell with `USE_DRIVE`, imports, device, data loading with the SHA-256 fingerprint), then:

1. **Introduction** (markdown): the BNN, the posterior, HMC, and the link to the L1 training, condensed from sections 2 and 3.
2. **Switches** (defaults shown): `LIKELIHOOD = 'laplace'; WARM_START; N_WARMUP = 500; N_SAMPLES = 1000; THIN = 1; N_LEAPFROG = 100; TARGET_ACCEPT = 0.8; N_CHAINS = 1; DATA_FRACTION = 1.0; SEED = 42`. The run folder is `DIRNAME` = `BNN_<likelihood>`.
3. **Model and parameters**: build `HEFTNet`, load the warm start, and print `ParameterView.summary()`, i.e. table 1 as found by the code.
4. **Posterior**: prior, likelihood, `Posterior`; time one gradient; print the predicted run time.
5. **Sample**: `HMC(...).run(...)`, with a progress line per trajectory (acceptance so far, ε, U, b); `RESUME = True` continues an interrupted run.
6. **Diagnostics** (section 7.2): traces, acceptance, autocorrelation, effective sample size, $\hat{R}$ if several chains.
7. **Results**: the BNN mean against POWHEG on the quick test rows (the scatter plot of `heft_train`); a few test spectra with the posterior mean and 68% and 95% bands; the 23 coefficient functions $a_i(m_{hh})$ with their bands; and the BNN mean compared with the warm-start L1 model.

7 Validation

7.1 The sampler on problems with known answers

Before `HEFTNet`, `code/hmc_toy.ipynb` (seconds on a CPU) runs:

1. **Bayesian linear regression**, a "network" with one linear layer, a Gaussian prior of fixed precision and Gaussian noise of known variance. The posterior is then Gaussian with a mean and covariance that can be written down. The sampled means and covariances must agree with them within their Monte Carlo errors (computed from the effective sample size). This tests the leapfrog integrator, the accept/reject step, the mass matrix and the bookkeeping of `ParameterView`.
2. **Neal's one-dimensional example** [1]: a small network fitted to noisy points of a smooth curve on part of an interval, with the hierarchical prior and Gibbs updates switched on. The posterior band should hug the data where there are data and widen outside. This is a visual check of the whole machinery, and a good teaching example.

7.2 Diagnostics for the `HEFTNet` run

- **Acceptance rate** in the sampling phase between about 0.6 and 0.9, and no (or very few) divergences.
- **Stationarity**: the traces of U, b and the τ_g show no drift after warm-up.

- **Mixing**: the autocorrelation of U, and of the prediction at a handful of monitor points (e.g. the first bin above threshold and a bin near 1 TeV for the Standard Model point), and from it the effective sample size; we aim for at least 100.
- **Several chains**, if run: Gelman–Rubin $\hat{R} < 1.05$ for the same quantities.

7.3 What counts as success

1. The toy checks pass.
2. The diagnostics above are met for `HEFTNet`.
3. The BNN mean is at least as accurate on the holdout spectra as the warm-start L1 model (median χ^2/N in `heft_test`), as expected of an average over nearby good fits.
4. The bands are informative: they are wider where the model is known to be weak (near threshold) than where it is good. A coverage check (how often POWHEG's holdout value falls inside the predictive interval) is reported, with the caveat of the watch box in section 2.3 for the single-scale likelihoods.

8 Moving the module to `drwho`

`hmc.py` will import nothing from `dihiggs`. Moving it means copying it to `drwho/drwho/hmc.py`, adding `"hmc"` to the list of lazily loaded submodules in `drwho/__init__.py`, and bumping the version, following the release protocol of `docs/drwho-scheme.pdf`. In `dihiggs`, `dihiggs/hmc.py` can then shrink to one line, `from drwho.hmc import *`, so the notebooks keep working; or the notebooks can import from `drwho` directly. The `drwho` commit is yours.

9 Not in this plan

- No change to `heft_train.ipynb`, `heft_test.ipynb`, `heft_checks.ipynb`, `heft_export.ipynb`, `heftnet.py` or the data.
- A BNN version of `heft_test.ipynb` (band-aware χ^2, coverage per m_{hh} bin) and an export of the ensemble to numpy/C++ are natural next steps, to be planned after we see the first results.
- The No-U-Turn sampler [4], which chooses L automatically, is not Neal's method and is left out; it could be added later.
- Stochastic-gradient samplers [6] and "cold posteriors" (raising the likelihood to a power [7]) are left out: they do not sample the posterior of eq. (1).
- Nothing is committed to git.

10 Decisions for you

Decisions before I proceed

1. **Likelihood.** Laplace as the default, Gaussian and Poisson (experimental) as options. Agreed?
2. **Prior.** Neal's hierarchical Gaussian, one τ_g per tensor (16 groups), Gamma hyperpriors of shape $\alpha_g = 1$ with prior mean $1/\sigma_g^2$ as in table 1, Gibbs updates after every trajectory. Agreed, or would you prefer fixed prior widths for a first version?
3. **Warm start** from `runs/L1Loss_ids/` (your `runs/L1Loss/` would also do; it is only read, never written). Agreed?
4. **Chains.** One chain by default, with `N_CHAINS` available; or should the default be 4 chains, for $\hat{R}$, on the GPU?
5. **File names.** `runs/BNN_<likelihood>/samples/params_0001.pth`, four digits. Acceptable?
6. **Toy notebook** `code/hmc_toy.ipynb`: include it? (Recommended: it is the only check that the sampler is right, and it would serve your students.)
7. **Location of this document**: `docs/bnn/bnn_plan.pdf`?

References

[1] R. M. Neal, *Bayesian Learning for Neural Networks*, Lecture Notes in Statistics 118, Springer (1996).

[2] R. M. Neal, "MCMC using Hamiltonian dynamics", in *Handbook of Markov Chain Monte Carlo*, eds. S. Brooks *et al.*, Chapman & Hall/CRC (2011), arXiv:1206.1901.

[3] S. Duane, A. D. Kennedy, B. J. Pendleton and D. Roweth, "Hybrid Monte Carlo", Phys. Lett. B **195** (1987) 216.

[4] M. D. Hoffman and A. Gelman, "The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo", J. Mach. Learn. Res. **15** (2014) 1593.

[5] P. Izmailov, S. Vikram, M. D. Hoffman and A. G. Wilson, "What are Bayesian neural network posteriors really like?", ICML 2021, arXiv:2104.14421.

[6] T. Chen, E. B. Fox and C. Guestrin, "Stochastic gradient Hamiltonian Monte Carlo", ICML 2014, arXiv:1402.4102.

[7] F. Wenzel *et al.*, "How good is the Bayes posterior in deep neural networks really?", ICML 2020, arXiv:2002.02405.